

Maintenance Lab

A regulatory update, from the spec to the review

Track 1, Lesson 6, hands-on session

The software we maintain

Rimborsi Dipendenti

An internal HR tool. Employees file expense reimbursement requests, the app computes the tax-exempt part and the taxable part.

New request: file a trip, meal, mileage or lodging expense

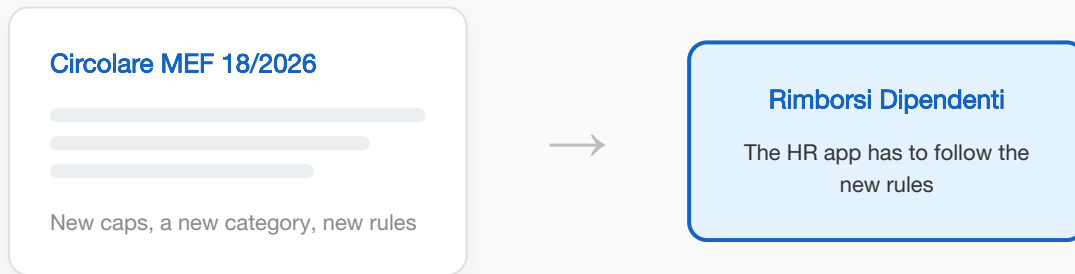
List: every request with its exempt and taxable amount

Monthly summary: the plafond used per employee

Current rules: the caps in force today

A Flask app, code in Italian, about 600 lines, with a pytest suite. The rules it applies come from a circular.

A new circular just landed



Ordinary maintenance: the rules changed, and the software has to change with them.

Two ways to take part

TRACK A

git + GitHub account

Fork, commit, push, CI on your fork. The full flow.

TRACK B

no git, no GitHub

Download the ZIP, work fully local. No CI.

The same code is produced either way.

The day, chapter by chapter

1 · Setup

10 min

2 · Read

20 min

3 · CLAUDE.md

20 min

4 · Try the app

10 min

5 · First change

demo

▼ break 15 min

6 · Plan

15 min

7 · Implement

15 min

8 · Tests

20 min

9 · Review

20 min

10 · Q&A

open

second break 10 min, between Implement and Tests

1

2

3

4

5

6

7

8

9

10

PHASE 1 / 10

1. Setup

Getting the codebase ready · 10 min

DO NOW

Get the codebase into your development environment and open it with Claude Code.

GOAL

The project is open and ready to work on.

Using git? Fork the repository and check out the `main` branch. · Track B: unzip the project and open the folder.

1

2

3

4

5

6

7

8

9

10

PHASE 2 / 10

2. Read the circular and ask Claude

Lesson 1, prompting · 20 min

DO NOW

Read the circular. In parallel, ask Claude how the app works: structure, the path of a request, where the rules live.

GOAL

You can say what changed in the rules and where the code applies them.

Ask with structure: role, context, task, output format. · Claude Code reads the circular PDF from the filesystem.

1

2

3

4

5

6

7

8

9

10

PHASE 3 / 10

3. Write CLAUDE.md

Lesson 2, project memory · 20 min

DO NOW

Write the project `CLAUDE.md` : what the app does, how it is organized, how to run it, the conventions to follow.

GOAL

A short `CLAUDE.md` that lets an agent work here without reading all the code.

Start with `/init` : it drafts a first version from the codebase. Then iterate, keeping only what really helps.

1

2

3

4

5

6

7

8

9

10

PHASE 3 / 10

Share your CLAUDE.md in chat

Paste your CLAUDE.md in the chat

We compare a few of them together: what people chose to keep, what they cut.

1

2

3

4

5

6

7

8

9

10

PHASE 4 / 10

4. Try the app

Lesson 2, working with the codebase · 10 min

DO NOW

Get the app running. Ask Claude to start it locally and help if something breaks. File a request, look at the list and the summary.

GOAL

You have used the app and seen how it behaves before changing it.

No local setup? Try the online example instead: `rimborsi-dipendenti-esercizio-6i39324wq.vercel.app/richieste`

1

2

3

4

5

6

7

8

9

10

PHASE 5 / 10

5. First change, together

Demo: the teacher plans and applies the first change · Lesson 5, plan mode

THE FLOW

1. Enter plan mode (Shift+Tab twice)
2. Launch the prompt
3. Read the plan, ask for fixes
4. Leave plan mode, let it run

PROMPT

```
We just received Circolare MEF 18/2026
(ENG_circolare_mef_18_2026.pdf). Apply only the first
change: the updated daily caps and the new monthly plafond,
effective 01/01/2026. Before editing anything, give me a
plan: which files change, the impact on the existing tests,
and the new tests you would add. Do not touch the stored
requests in data/.
```

Reading and fixing the plan happens before any code is written. That is the point of the demo.



BREAK

15 minutes

1

2

3

4

5

6

7

8

9

10

PHASE 6 / 10

6. Plan the rest

Lesson 5, plan mode · 15 min

DO NOW

In plan mode, plan the remaining changes that finish updating the software. Order them so the tests stay green at each step.

GOAL

A reviewed plan that includes the transitional regime.

Plan mode: explore and plan, no edits yet. · Read the plan and ask for fixes before you leave it.

1

2

3

4

5

6

7

8

9

10

PHASE 6 / 10

Demo: the teacher plans it live

Now I plan the rest in front of you

Watch how the plan is read, questioned, and corrected before any code is written.

1 2 3 4 5 6 **7** 8 9 10 PHASE 7 / 10

7. Implement step by step

Lessons 2 and 3, working with the agent · 15 min

DO NOW

Implement the plan one change at a time. Test after each step, and commit if you use git.

GOAL

The changes are in, tests green, the 2025 dates still working.

One change at a time, not all at once. · If the agent goes off the plan, stop it and bring it back.

1

2

3

4

5

6

7

8

9

10

PHASE 7 / 10

Demo: the teacher implements it live

Now I implement one change in front of you

One change, a test, a check in the app. Then the next one.



BREAK

10 minutes

Feedback time

**Please fill out the survey
and give us your feedback**



*Scan the QR code to see the
survey*

8. Tests

Lessons 3 and 4, tests and pipeline · 20 min

DO NOW

Write a test for every case in Section 6 of the circular. Track A pushes and watches CI; Track B runs `pytest` locally.

GOAL

Section 6 covered, CI green on the fork (Track B: green locally).

Expected values come from the circular, written as literals. · If a test fails, the spec is right, so fix the code.

1

2

3

4

5

6

7

8

9

10

PHASE 9 / 10

9. Review with a subagent

Lesson 4, subagents · 20 min

BUILD THE REVIEWER WITH CLAUDE

1. Ask Claude to create a review subagent for you
2. Describe the role: domain reviewer for tax rules
3. Set the focus: numbers vs the circular, edge cases, UI and backend in sync
4. Save it under `.claude/agents/`

THEN RUN IT

Run the subagent on your change, with the circular as the reference. It starts with no prior memory of your session.

GOAL

At least one real finding handled, with a reason for what you skip.

1 2 3 4 5 6 7 8 9 10 REVIEW

How did the review go?

What did the subagent find?

Real findings, false positives, and what you decided to fix or skip.

1 2 3 4 5 6 7 8 9 10 PHASE 10 / 10

10. Q&A and open discussion

How did it go? · Lesson 5, DORA metrics as a lens

How much time went into understanding the spec, compared to writing code?

Did any change break the tests?
How long did it take to recover?

Where did the AI help most, and where did it need close control?

Lead time, deployment frequency, change failure rate, time to restore: the four DORA metrics, as a way to read the day.

What comes next

FINAL PROJECT

The course ends with a project of your own. The last three meetings are dedicated to it.

NEXT TIME, 26 JUNE

Each of you presents your idea in a few minutes and gets feedback from the teacher on how to approach it.

Come to the next meeting with an idea already in mind.

Thank you

Track 1, Lesson 6